

Reproduction Report

MitsuhaLe

May 8, 2026

1 Overview

I reproduced the block Newton method for nonlinear eigenvalue problems, which was proposed by D. Kressner. The original paper is *A block Newton method for nonlinear eigenvalue problems* (<https://doi.org/10.1137/15M1049426>). Overall, the algorithm is rather concise and implementation-friendly. It also exhibits local quadratic convergence. I spent roughly one week understanding and implementing the algorithm. The code is available at <https://github.com/MitsuhaLe/nep-block-newton-matlab>. During the reproduction process, I made some severe minor mistakes, which are presented in the following sections.

It is worth noting that I also ask for help from AI tools, such as ChatGPT and Copilot.

2 Phase 1: Understanding the Algorithm

After reading the paper carefully, I found that the algorithm is quite straightforward. The main function has already been provided in the paper, i.e. `nlevp_newtonstep` in Appendix A:

Listing 1: `nlevp_newtonstep` function from the original paper.

```
1 function [dX, dS] = nlevp_newtonstep( A, f, X, S, W, RT, RV )
2 % Computes the solution (dX,dS) to the linearized system in one
3 % step of
4 % the block Newton method for a nonlinear eigenvalue problem (NLEVP).
5 % Input: A - 3d-array containing the matrices A_j of the NLEVP
6 %         f - handle to a function f(j,M) that returns f_j(M)
7 %         for any square matrix M
8 %         (X,S) - current iterate. S is assumed upper triangular.
9 %         W - 3d-array containing the normalization matrices W_j
10 %         (RT,RV) - right-hand side
```

The remaining tasks were to implement algorithm 2 in the paper for setting up an initial pair (X_0, S_0) and the Armijo rule.

3 Phase 2: Implementing the Algorithm

My target platform was MATLAB. At first, I implemented the algorithms without a clear structure. I only hope that the code would run and give the correct output. Unfortunately, I made quite a few mistakes in coding. Meanwhile, I also doubted the effectiveness of the block newton method itself, since it is only a locally convergent method. So I wasted a lot of time on debugging and trying to make the algorithm work.

I spent an entire day working only in MATLAB. It seemed that I was still far from a correct implementation. Then I searched for some existing implementations online. I found a Julia implementation in <https://github.com/NEP-Pack>. There is an example of the "gun" problem, which is a standard test problem for nonlinear eigenvalue problems. I tried to run the

example, and it really worked. I was impressed by the performance of the code and the language Julia itself. Then I told myself that I should try to understand the code and implement it in MATLAB.

4 Phase 3: Debugging and Mistakes

4.1 Mistakes 1

NEP-PACK provides the following code for the "gun" benchmark problem:

Listing 2: Example code listing.

```
1 nep=nep_gallery("nlevp_native_gun");
2 n=size(nep,1)
3 S=150^2*[1.0 0; 0 1]; V=[[1 0; 0 1]; zeros(n-2,2)];
4 (Z,X)=blocknewton(nep,S=S,X=V,logger=1,armijo_factor=0.5,maxit=20)
```

After 14 iterations, the error is 2.525942e-15.

Note the above codes call the function `blocknewton`. I wanted to debug with this function, in order to inspect the intermediate outputs. However, I found all my debugging attempts failed. I was really confused. I doubted that I was not familiar with the language Julia, which made debugging difficult for me. Then with the help of AI tools, I found that what I changed was the local file named `method_blocknewton.jl`, while the code calling the function `blocknewton` is calling the function in the NEP-PACK package. This is *mistake 1*. I realized that I have to integrate the function `method_blocknewton` into my code.

4.2 Mistake 2

After fixing the above mistake, I obtained a working file named `testgun.jl`. Then I compared this implementation in Julia with my MATLAB code line by line. I found that the two codes are quite similar, except for some details. In MATLAB, I used some functions, such as `exp`, `sqrt`, for matrix functions. However, these functions operate entrywise on matrices, which is not what I actually want. A senior labmate told me that I should use `expm`, `sqrtn` for matrix functions. This is *mistake 2.1*. Besides, I used $A - 2$ to denote $A - 2I$, which is also wrong and should be $A - 2 \times \text{eye}(\text{size}(A))$. This is *mistake 2.2*.

5 Phase 4: Final Results

I successfully implemented the block newton method in MATLAB. I tested it on the "gun" problem. It worked well but it was much slower than the Julia implementation. I used MATLAB's profiler to find the bottleneck, and it turns out that solving linear systems are the main time-consuming part. The results are shown in the following pictures.

39	[dX, dS] = nlevp_newtonstep(A, X*U...	7	162.670	99.9%	
64	figure(2)	1	0.060	0.0%	
42	[j, tau] = armijoRule(A, X, S, dX, ...	7	0.053	0.0%	
65	semilogy(Res)	1	0.046	0.0%	
32	[RT, Res(iter)] = resT(A, X, S);	7	0.019	0.0%	
56	sol = [T11 T12; T21 T22] \ [RT(:,i); RV(:,...	14	142.738	87.7%	
36	T11 = T11 + A{j}*f(j,s); % matrix T(s)	56	19.086	11.7%	
34	T11 = zeros(n);	14	0.383	0.2%	
73	end	14	0.350	0.2%	
42	T12 = T12 + A{j}*X*DF(1:k,k+1:2*k); % [DF(...	56	0.025	0.0%	

Figure 1: Profiler results.

6 Summary and Conclusions

I don't have much experience in reproducing papers before. This project tells me that reproduction studies is very meaningful. It can help me understand the algorithm deeply, and even give me some insights into the algorithm and the problem(here, NEP).

I need to get more coding practice, and also learn more about the language Julia. I think Julia is a very good language for scientific computing, and I may try to use it more in the future. Compared with MATLAB, Julia is more complicated and less convenient, since its development ecosystem is less mature than MATLAB's.

Next time, I will try to avoid the mistakes I made in this project, and also try to make my codes more concise and structured.